Project Imperium

Pod K | Office Hours (10th March)

Run time: **30-45** minutes

ようこそ

Bem-vindo 欢迎 Hoşgeldiniz

ברוך הבא **Welcome!** स्वागत

Bienvenido مرحباً

Willkommen

This session is being transcribed and minutes will be made available on the meeting dashboard.

Project Imperium

Pod K | Office Hours (10th March)

Run time: **30-45** minutes

Housekeeping

- *Infrastructure issues affecting supply:* QC throughput slowed because the automation agent that sources and packages Phase 1 tasks from open-source projects is currently experiencing infrastructure issues, which temporarily reduced the rate at which new tasks enter the pipeline.
- *Automation improvements and cron job rollout:* To improve throughput, parts of the Phase 1 review layer have been automated. Starting today, a cron job will reroute tasks every hour, automatically moving them through the review pipeline to reduce backlog and speed up QC processing.
- *Auto-QC system benefits:* High-scoring tasks will be automatically merged and treated the same as normal merged work for compensation, while low-scoring tasks will be automatically returned for revision. This is designed to reduce review delays and process submissions faster while maintaining quality.

This session is being transcribed and minutes will be made available on the meeting dashboard.

Project Imperium

Presentation Agenda

Technical Agenda

1. **Phase 2: Failure Modes**
   Knowledge Base discoverability, trajectory, and expert contribution issues

2. **Bad Submission Example**
   Real rejected submission: fabricated robustness, wrong-domain style, missing coverage

3. **Good Submission Example**
   Real STRONG PASS: genuine gaps, decisive answers, tight alignment

Hosts

Oliver, **EPM**.
Gabriel, **Super Reviewer**.
Harsh, **Tech Lead**.

This session is being transcribed and minutes will be made available on the meeting dashboard.

Phase 2: Common Failure Modes

Knowledge Base & Trajectory: Discoverability, Expert Value & Alignment

11 documented failure modes | #1 cause of rejection: Discoverable questions

Phase 2: Failure Frequency & Severity

#	Failure Mode	Severity	Frequency	Primary Cause
1	KB: Discoverable Questions	FAIL	Very High	Documenting existing code patterns
2	KB: Meta File Restating	FAIL	Very High	Not reading meta files first
3	KB: Insufficient Expert Contribution	FAIL	High	Unrefined LLM output
4	Trajectory: Discoverable Criteria	FAIL	High	Mirrors KB discoverability
5	KB: Structural Format Violations	FAIL/MINOR	Moderate	LLM-generated scaffolding
6	KB: Factual Inaccuracy	HARD FAIL	Moderate	Confident but wrong claims
7	Alignment Gaps (KB to Trajectory)	FAIL	Moderate	Deliverables written independently
8	Trajectory: Language Violations	FAIL	Low-Mod	Not following format rules
9	KB: Test Leakage	HARD FAIL	Low	Referencing test values

This session is being transcribed and minutes will be made available on the meeting dashboard.

VERY HIGH FREQUENCY

#1 KB: Discoverable Questions

The single most common failure mode in Phase 2. Observed in the majority of reviewed submissions.

Questions whose answers are directly observable by reading the existing codebase and task files. When the prompt says 'implement similar to X', everything observable in X's implementation is discoverable.

BAD (Discoverable)

"How should the Python wrapper synchronize with the hardware driver?"

The existing front_face property demonstrates exactly this pattern.

GOOD (Non-discoverable)

"Does the cull_face setter need to gate its dispatch on the current state of the enable flag?"

No existing property has an enable-flag interaction.

The Discoverability Test

Before writing each question, ask: "Where in the codebase or task files would a developer find this answer?" If you can point to a specific line or file, the question is discoverable. Replace it. Focus on areas where the codebase is SILENT, AMBIGUOUS, or where the new feature DIFFERS structurally from existing patterns.

Phase 2: KB & Trajectory Pitfalls

Meta File Restating — FAIL

Answers stated verbatim in requirements.json, interface.md, problem_statement.md, or prompt_statement.md. Search meta files before writing.

Test Leakage — HARD FAIL

References to test.patch, test assertions, test file names, or exact test values. Describe general behavioral categories, not exact asserted values.

Insufficient Expert Contribution — FAIL

KB reads like polished LLM output. Indicators: generic questions, hedging answers, em-dashes throughout. At least 2-3 questions must target genuinely tricky parts.

Trajectory: Discoverable — FAIL

Criteria answerable by reading task files or codebase. If the code shows the pattern clearly, it is discoverable. Also fails if too generic for any task.

Factual Inaccuracy — HARD FAIL

Answers contradict actual requirements, golden patch, or test expectations. Cross-reference every factual claim against the golden patch.

Trajectory: Language Violations — FAIL

Must start with 'Should have clarified/asked/verified'. One topic per criterion. No impl details, no test refs. Exactly 2-3 criteria.

Bad Submission: Oppia Contributor Dashboard

Verdict: MAJOR FAIL -- Sent back to writer

Prompt, problem statement, interface, and requirements were all PASS.
The entire failure was in rubric.json.

Bad Example: Functional Criteria Failures

Issue 1: Wrong Subheading String (functional-5)

Rubric says: 'Columns whose value is numeric zero, empty string, boolean false, or empty list are included in the rendered table output.'

test.patch asserts:

category: '' (empty string) -- ABSENT from expected outputs tags: [] (empty list) -- ABSENT from expected outputs
exposedMachines: 0 (numeric zero) -- PRESENT in expected outputs
publicExploit: False (boolean false) -- PRESENT in expected outputs

Claims four value types are "included" but test evidence proves empty strings and empty lists are excluded.

Only numeric zero and boolean false are actually kept. (hence, description does not match what the mapped test assertion actually checks.)

Issue 2: Missing Endpoint Coverage

test.patch adds 2 new endpoint tests:

- Deleted skill opportunity endpoint test
- Deleted exploration opportunity endpoint test
- requirements.json entries 5-7 describe this behavior.

But ZERO rubric criteria cover the endpoint behavior. Any test assertion without a matching rubric criterion is a gap.

Bad Example: Fabricated Robustness Criteria

All 3 robustness criteria were completely invented -- NONE exist in test.patch, golden.patch, or any task file.

robustness-1

Weight: Major

"'No Data Received' fallback message"

This feature appears NOWHERE in test.patch, golden.patch, or any task file. Marked weight 'major', meaning it would fail any correct solution.

robustness-2

Weight: Minor

"Pagination for >10 results"

Completely invented. Zero hits for 'pagination' across all project files. This feature does not exist in the task.

robustness-3

Weight: Minor

"Page link buttons for navigation"

Also invented. Depends on robustness-2 which itself doesn't exist. Cascading fabrication.

Bad Example: Copy-Pasted Style Criteria

style-1: snake_case naming

Examples given:
`'particle_set, lagrange_values'`

Problem(s):

- These are from a physics/math codebase, not Oppia's education platform. Clear copy-paste from another task.

style-2: PascalCase classes

Examples given:
`'GridData, ParticleCollider'`

Problem(s):

- Nothing to do with this task
- Golden patch introduces no new classes
- Rationale mentions 'const, let' (JavaScript) while claiming Python PEP 8

Lesson

Style criteria must use examples FROM THE ACTUAL TASK. Cross-domain examples are an instant red flag for copy-paste. Also, 8 descriptions were missing terminal periods -- inconsistent formatting signals lack of care.

Good Submission: axlearn Einops Module

Verdict: STRONG PASS -- Zero fixes needed. Approved on first review.

Every KB question targeted a genuine gap in the specification.
Trajectories tightly aligned with KB, decisive language, concrete failure modes.

Good Example: Knowledge Base (6 Questions, All Non-Discoverable)

Axis splitting: N-1 inference rule
Non-discoverable

Absent from all meta files. requirements.json says 'axis size kwargs' but never specifies inference logic.

Merge ordering inside parenthesized groups
Non-discoverable

Highest-value question. Meta files silent on whether intra-group reordering is permitted. Aligns with trajectory-1.

Split-axis output ordering freedom
Non-discoverable

Distinct from Q2 (merge vs split). The constraint on split-axis ordering is not stated in any meta file.

Selective singleton dimension handling
Borderline OK

Core expand/squeeze in requirements, but selective combination aspect adds marginal value. Weakest but acceptable.

Repeat: existing axis ordering constraint
Non-discoverable

Ordering constraint absent from all meta files. 'ValueError' for reordering is in error context, not behavioral spec.

New axis placement within groups
Non-discoverable

Subtle interleaving behavior not covered by any meta file. Position before vs after produces different outputs.

Good Example: Trajectory Criteria (3 Non-Discoverable)

trajectory-1 **major** (Maps to Q2)

Should have clarified whether axes within a parenthesized output group can appear in a different order than their original input ordering when merging dimensions.

Rationale: Merging axes in a different order than they appear in the input produces a fundamentally different data layout in the resulting tensor. Assuming axes must stay in their original order within groups leads to silent data corruption.

trajectory-2 **major** (Maps to Q5)

Should have asked whether the repeat operation permits reordering existing input axes in the output pattern or only allows inserting new axes while preserving existing axis order.

Rationale: Repeat and rearrange have different constraints on existing axes, but the task description does not distinguish them. Failing to clarify leads to either incorrectly allowing transposing during repeat or incorrectly rejecting valid patterns.

trajectory-3 **minor** (Maps to Q6)

Should have asked how the placement of a new repeat axis relative to an existing axis within a parenthesized output group affects the resulting tensor layout.

Rationale: Placing a new axis before versus after an existing axis within a parenthesized group produces different interleaving patterns in the merged dimension. Treating both orderings identically produces incorrect results.

Why this submission *works*

It’s simple, really.

1 Every question found a real gap
Writer verified each Q against all meta files. Zero questions restate discoverable content.

2 Answers are decisive, not hedging
No 'this depends on implementation'. Example Q5: 'Reordering existing axes is not allowed.' Clear behavioral directive.

3 Trajectories are tightly aligned with KB
T1 maps to Q2, T2 maps to Q5, T3 maps to Q6. No orphan trajectories, no orphan KB questions.

4 Rationales explain what goes wrong
Not 'this is important for correctness.' Instead: 'silent data corruption,' 'incorrectly allowing transposing during repeat.'

5 Language is clean throughout
No agent references, no test references, no meta file name references. No em-dashes. Clean formatting.

This session is being transcribed and minutes will be made available on the meeting dashboard.

Side-by-Side: Bad vs Good Submissions

Aspect	Bad (Oppia / imgaug)	Good (Defender ATP / axlearn)
Functional Criteria	Missing test coverage, wrong strings	Each maps to a test assert, correct strings
Robustness Criteria	Fabricated features (pagination, fallback)	Edge cases of implemented behavior
Style Criteria	Copy-pasted from unrelated task	Task-specific (docstrings, naming)
KB Questions	9 of 11 restate meta files	All non-discoverable, genuine gaps
KB Answers	Restates spec with filler	Decisive behavioral directives
Trajectory Alignment	2 of 3 discoverable, weak rationales	All map to KB, concrete failure modes
Review Outcome	**MAJOR FAIL / REJECT, 14 failures**	**PASS / STRONG PASS, 0 fixes**

This session is being transcribed and minutes will be made available on the meeting dashboard.

Key Takeaways

Remember these key takeaways.

- Before writing your rubric, read test.patch line by line. Every assert is a potential criterion. Every criterion must trace back to an assert.
- Before writing any KB question, search all 4 meta files for the answer. If you find it -- even paraphrased -- throw the question out.
- Never fabricate features. If it is not in test.patch or golden.patch, it does not belong in the rubric.
- Write decisive answers and concrete rationales. Not 'this is important' -- explain what goes wrong.
- Check alignment: every trajectory maps to a KB question, every rubric criterion maps to a test assertion.

Q&A?

This session is being transcribed and minutes will be made available on the meeting dashboard.